

Deep Learning

Artificial Intelligence

Jacopo Parretti

II Semester 2025-2026

Contents

I	Introduction to Deep Learning	6
1	What is Deep Learning?	6
1.1	Positioning within AI	6
1.2	Why Deep Learning?	6
1.3	Deep Learning as Representation Learning	6
1.4	Why Representation Matters	7
2	A Brief History of Neural Networks	7
3	The Perceptron	7
4	Activation Functions	8
5	From Perceptron to Neural Networks	9
5.1	The Multi-Layer Idea	9
5.2	Dimensions	9
5.3	Multilayer Perceptron (MLP)	9
6	Training Neural Networks	10
6.1	Training and Testing Stages	10
6.2	Forward and Backward Pass	10
7	Loss Functions	10
7.1	Standard Loss Functions	10
7.2	Cross-Entropy and Information Theory	10
7.3	Loss as an Optimization Objective	11
8	Backpropagation and Gradient Descent	11
8.1	General Idea	11
8.2	Batch Gradient Descent	12
8.3	Stochastic Gradient Descent (SGD)	12
8.4	Mini-Batch Gradient Descent	12
II	Backpropagation: Derivation and Training Dynamics	13
9	Expressive Capacity of Multilayer Networks	13

9.1	What a Single Perceptron Can Represent	13
9.2	Capacity with Hidden Layers	13
10	Formal Derivation of Backpropagation	13
10.1	Setup and Notation	13
10.2	Derivatives for the Output Layer	14
10.3	Derivatives for the Hidden Layer	14
10.4	General Weight Update Rule	15
10.5	Algorithm	15
10.6	Complexity	15
11	Optimization Challenges	16
11.1	Learning Rate Sensitivity	16
11.2	Local Minima	16
11.3	Saturation	16
12	Generalization	16
12.1	Definition	16
12.2	Overfitting	16
13	Cross-Validation Strategies	17
13.1	Resubstitution	17
13.2	Holdout	17
13.3	Averaged Holdout	17
13.4	Leave-One-Out (LOO)	17
13.5	k -Fold Cross-Validation	17
14	When to Use Neural Networks	18
III	Convolutional Neural Networks	18
15	Regularization	18
15.1	The Bias-Variance Trade-off	18
15.2	Early Stopping	18
15.3	L2 Regularization (Ridge / Weight Decay)	19
15.4	Dropout	19
15.5	Batch Normalization	19
16	Convolutional Neural Networks	19
16.1	Motivation: Receptive Fields and Weight Sharing	19
16.2	The Convolution Operation	20
16.3	Feature Maps and Filters	20
16.4	Non-linearity	20
16.5	Pooling	20
16.6	Fully Connected Layer and Softmax	21
16.7	Standard Architecture Pattern	21
17	Landmark Architectures	21
17.1	LeNet (1998)	21
17.2	AlexNet (2012)	22
17.3	ZFNet (2013)	22
17.4	VGGNet (2014)	22

17.5 GoogLeNet / Inception (2014)	23
17.6 ResNet (2015)	23
17.7 Performance Summary	24
18 Transfer Learning	24
18.1 ConvNet as Fixed Feature Extractor	24
18.2 Fine-tuning	25
19 Beyond Supervised Classification	25
19.1 Autoencoders	25
19.2 Generative Adversarial Networks	25
19.3 Recurrent Networks and LSTMs	26
19.4 Transformers	26
IV Image Classification	26
20 The Image Classification Problem	27
20.1 Task Definition and Applications	27
20.2 Statistical Learning Framework	27
21 Classic Recognition Pipeline	28
22 Classification Fundamentals	28
22.1 Binary and Multiclass Classification	28
22.2 Classification Methods	28
23 Neural Networks for Classification	29
23.1 Linear Machine	29
23.2 Two-Layer and Three-Layer Networks	29
24 Softmax Classifier and Cross-Entropy Loss	30
24.1 From Scores to Probabilities	30
24.2 Cross-Entropy Loss	30
24.3 Dataset Loss	30
25 CNN Spatial Arithmetic	31
25.1 Stride and the Integer-Fit Constraint	31
25.2 Zero Padding	31
25.3 1×1 Convolutions	31
25.4 Translation Equivariance	32
25.5 Receptive Field	32
26 Training Best Practices	33
26.1 Train / Validation / Test Split	33
26.2 Training Dynamics	33
V Recurrent Neural Networks and LSTM	33
27 Sequential Data and Problem Formulation	33
27.1 Dealing with Sequential Data	33
27.2 Approaches to Sequence Modelling	33
27.3 Problem Formulations	34

27.4 Representing Text: Word Embeddings	34
27.5 Why Fixed-Window Approaches Fail	34
28 Recurrent Neural Networks	35
28.1 The Recurrence Relation	35
28.2 Key Properties	35
28.3 Unrolling Over Time	35
28.4 Loss and Backpropagation Through Time (BPTT)	35
28.5 Gradient Pathologies	36
28.6 RNN Applications	36
29 Long Short-Term Memory (LSTM)	36
29.1 Motivation	36
29.2 Gated Cells	36
29.3 LSTM Cell State	36
29.4 The Four Steps of an LSTM Cell	37
30 Variants	37
30.1 Gated Recurrent Unit (GRU)	37
30.2 Transformers	38
VI Attention Mechanism & Transformers	38
31 From RNNs to Transformers	38
31.1 The Historical Context	38
31.2 Problems with RNNs/LSTMs	39
31.3 Transformers: Key Advantages	39
32 Transformer Architecture	39
32.1 Encoder–Decoder Structure	39
33 Encoder in Detail	40
33.1 Components	40
33.2 Input Embeddings	40
33.3 Positional Encoding	40
33.4 Stack of Encoder Layers	41
34 Self-Attention Mechanism	41
34.1 Query, Key, Value	41
34.2 Step-by-Step Computation	41
34.3 Matrix Form	42
35 Multi-Head Self-Attention	42
35.1 Motivation	42
35.2 Mechanics	42
36 Feed-Forward Network and Normalization	43
36.1 Pointwise Feed-Forward Network	43
36.2 Add & Norm Summary	43
37 Decoder in Detail	43
37.1 Structure	43
37.2 Masked Multi-Head Self-Attention	44

37.3 Encoder-Decoder Attention	44
37.4 Output Stack	44
38 Language Pre-Training	44
 VII Beyond Supervised Learning: Unsupervised Learning, Clustering & Self-Supervised Methods	 45
39 Types of Learning: A Taxonomy	45
39.1 Supervision Axes	45
39.2 Motivation for Unsupervised Learning	45
40 Self-Supervised Learning	46
40.1 Core Idea	46
40.2 Context Prediction (Doersch et al., ICCV 2015)	46
41 Learning by Clustering	46
41.1 General Framework	46
42 Progressive Unsupervised Learning (PUL)	47
42.1 Problem Setting: Unsupervised Person Re-Identification	47
42.2 PUL: Formulation	47
42.3 Algorithm 1: PUL	48
42.4 Self-Paced Learning Intuition	48
42.5 Semi-Supervised Extension	48
43 DeepCluster	48
43.1 Motivation	48
43.2 Pipeline	49
43.3 Architecture and Implementation Details	49
43.4 Why It Works	49

Part I

Introduction to Deep Learning

1 What is Deep Learning?

1.1 Positioning within AI

Deep Learning is a subset of machine learning and, more specifically, of **representation learning** (also called *feature learning*). The fundamental goal is to *learn underlying features directly from data*, eliminating the need for hand-engineered feature extractors.

The standard machine learning pipeline separates two concerns: **feature extraction**, designed by a domain expert, and **classification**, learned from labeled examples. Deep learning collapses this pipeline — the network performs both steps jointly, learning task-relevant representations end-to-end from raw input.

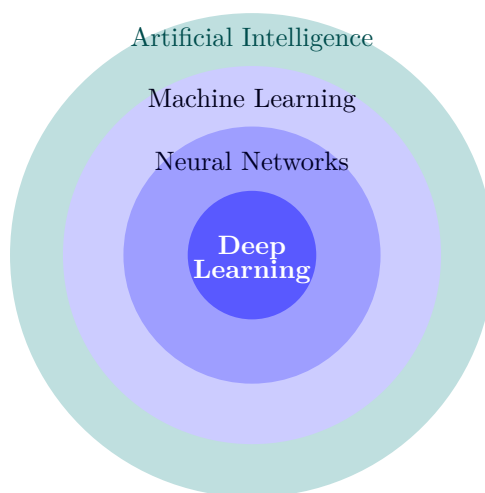


Figure 1: Deep Learning as a nested subset of AI.

1.2 Why Deep Learning?

A critical empirical observation motivates deep learning: as the volume of available data grows, most classical learning algorithms **plateau** in performance, while deep neural networks **continue to improve**. This favorable scaling behavior, combined with large-scale datasets and modern hardware accelerators, explains the field’s current dominance.

1.3 Deep Learning as Representation Learning

Deep learning is explicitly a form of **representation learning**: the network learns a hierarchy of representations, layer by layer, from raw input to task-relevant abstractions. In visual recognition, the hierarchy proceeds roughly as:

- **Visible layer**: raw pixel values
- **1st hidden layer**: low-level features (edges, color blobs)
- **2nd hidden layer**: mid-level features (corners, contours)
- **3rd hidden layer**: high-level features (object parts)

- **Output layer:** class identity (car, person, animal, ...)

The deeper the network, the more abstract and task-specific the learned representations.

1.4 Why Representation Matters

The choice of representation fundamentally affects the difficulty of learning. A canonical example: two point clouds that cannot be separated by a linear boundary in Cartesian coordinates (x, y) become trivially linearly separable after converting to polar coordinates (r, θ) . Deep networks learn such transformations automatically, avoiding the need for manual feature engineering.

2 A Brief History of Neural Networks

The current success of deep learning is recent, but its intellectual roots extend to the 1940s. Three factors explain the modern renaissance:

1. **Big data:** availability of massive labeled datasets (e.g., ImageNet, web-scale corpora).
2. **Improved software:** modern frameworks (PyTorch, TensorFlow) with automatic differentiation and GPU support.
3. **Hardware:** GPUs and TPUs provide the parallelism required to train large models in tractable time.

Key milestones (1943–present):

- **1943:** McCulloch & Pitts model a neuron with electrical circuits.
- **1949:** Hebb formalizes synaptic plasticity in *The Organization of Behavior*.
- **1956:** The Dartmouth Summer Research Project launches AI as a field.
- **1957–58:** Rosenblatt develops the **Perceptron** — the oldest neural network still in common use today.
- **1959:** Widrow & Hoff develop ADALINE and MADALINE — the first neural network applied to a real-world problem.
- **1969:** Minsky & Papert prove the Perceptron’s fundamental limitations (*Perceptrons*); neural network research stalls.
- **1981–82:** Hopfield presents energy-based associative memory; interest revives.
- **1985:** Backpropagation popularized (Rumelhart, Hinton, Williams).
- **1997:** LSTM proposed by Schmidhuber & Hochreiter.
- **1998:** LeCun publishes *Gradient-Based Learning Applied to Document Recognition*.
- **Now:** neural networks are ubiquitous; discussions about their future are prevalent.

3 The Perceptron

The **perceptron** is the basic building block of neural networks, introduced by Frank Rosenblatt in 1958. It is a computational abstraction of a biological neuron.

Definition 3.1 (Perceptron). A perceptron takes m real-valued inputs $x_1, \dots, x_m$ together with a constant bias w_0 , computes their weighted sum, and passes the result through an activation function g :

$$\hat{y} = g\left(w_0 + \sum_{i=1}^m x_i w_i\right)$$

In the original formulation, g is the Heaviside step function. Information flows strictly **forward**: from inputs to output, with no feedback connections.

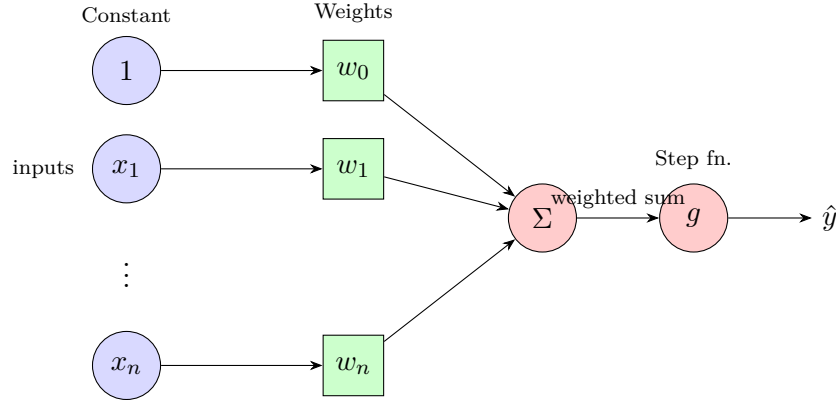


Figure 2: Perceptron: inputs are multiplied by learned weights, summed, and passed through an activation function.

4 Activation Functions

Activation functions introduce **non-linearity** into the network. Without them, stacking linear layers collapses to a single linear transformation, making the model incapable of representing non-linear decision boundaries. They are a crucial element of every architecture.

- **Sigmoid:** $\sigma(x) = \frac{1}{1 + e^{-x}}$
Output in $(0, 1)$; historically widespread but suffers from vanishing gradients at saturation.
- **Tanh:** $\tanh(x)$
Zero-centered; output in $(-1, 1)$; generally preferred over sigmoid in hidden layers.
- **ReLU** (Rectified Linear Unit): $\max(0, x)$
Sparse activations, computationally cheap; the default choice in modern architectures.
- **Leaky ReLU:** $\max(0.1x, x)$
Mitigates the dying ReLU problem by allowing a small gradient for negative inputs.
- **ELU** (Exponential Linear Unit):

$$\text{ELU}(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

Smooth for negative inputs; can push mean activations toward zero.

- **Maxout:** $\max(w_1^T x + b_1, w_2^T x + b_2)$
Generalizes ReLU and Leaky ReLU; learns the activation shape at the cost of doubling the parameter count.

5 From Perceptron to Neural Networks

5.1 The Multi-Layer Idea

The goal is to build a **larger, non-linear model**. The key idea: concatenate many perceptron systems and insert a **non-linear activation function** between computation layers. A two-layer network (one hidden layer) computes:

$$\mathbf{y}_1 = \mathbf{x}W_1 + \mathbf{b}_1, \quad \mathbf{y}_2 = \phi(\mathbf{y}_1)W_2 + \mathbf{b}_2, \quad \hat{\mathbf{y}} = s(\mathbf{y}_2)$$

where ϕ is a non-linear activation (e.g., ReLU) and s is the output activation (e.g., softmax for classification).

5.2 Dimensions

For a problem with F input features, H hidden neurons, and C output classes:

- $\mathbf{x} \in \mathbb{R}^{1 \times F}$, $W_1 \in \mathbb{R}^{F \times H}$, $\mathbf{b}_1 \in \mathbb{R}^{1 \times H}$, $\mathbf{y}_1 \in \mathbb{R}^{1 \times H}$
- $W_2 \in \mathbb{R}^{H \times C}$, $\mathbf{b}_2 \in \mathbb{R}^{1 \times C}$, $\mathbf{y}_2 \in \mathbb{R}^{1 \times C}$

Example: a 28×28 grayscale image ($F = 784$), $H = 1024$ hidden neurons, $C = 3$ classes:

$$\underbrace{\mathbf{x}}_{1 \times 784} \cdot \underbrace{W_1}_{784 \times 1024} + \underbrace{\mathbf{b}_1}_{1 \times 1024} = \underbrace{\mathbf{y}_1}_{1 \times 1024} \xrightarrow{\text{ReLU}} \underbrace{W_2}_{1024 \times 3} + \underbrace{\mathbf{b}_2}_{1 \times 3} = \underbrace{\mathbf{y}_2}_{1 \times 3} \rightarrow s(\mathbf{y}_2)$$

5.3 Multilayer Perceptron (MLP)

The **Multilayer Perceptron** (MLP) extends this to an arbitrary number of stacked hidden layers. For input $x = (\chi_1, \dots, \chi_p)^T$:

1. **Hidden layer** with weight matrix W^h and activation σ :

$$u_j = \sum_i W_{ji}^h \chi_i, \quad h_j = \sigma(u_j)$$

2. **Output layer** with weight matrix W^y :

$$v_k = \sum_j W_{kj}^y h_j, \quad y_k = \sigma(v_k)$$

The network is **fully connected**: every neuron in layer ℓ connects to every neuron in layer $\ell + 1$.

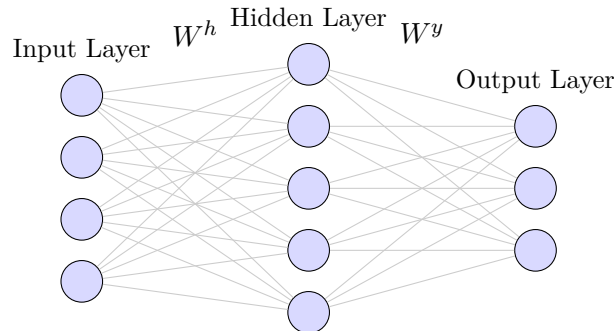


Figure 3: Multilayer Perceptron with one hidden layer.

6 Training Neural Networks

6.1 Training and Testing Stages

Neural network training proceeds in two distinct phases:

- **Training stage:** input data is passed through the network and compared to the ground truth. The network parameters are adjusted to minimize the discrepancy.
- **Testing stage:** new, unseen input is passed through the *trained* (frozen) network to produce a prediction (best guess).

6.2 Forward and Backward Pass

Each training step consists of two passes:

1. **Forward pass:** propagate the input through the network to compute a prediction.
2. **Backward pass (backpropagation):** compute the loss between prediction and ground truth; propagate the error signal backward through the network via the chain rule; update weights to reduce the error.

7 Loss Functions

A loss function quantifies the discrepancy between the network's prediction p and the ground truth t . The choice is **task-dependent**.

7.1 Standard Loss Functions

1. **Regression** — Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n} \sum (y - \hat{y})^2$$

2. **Binary classification** — Binary Cross-Entropy (BCE):

$$\text{BCE}(t, p) = -(t \cdot \log(p) + (1 - t) \cdot \log(1 - p))$$

3. **Multiclass classification** — Categorical Cross-Entropy (CCE):

$$\text{CCE}(p, t) = - \sum_{c=1}^C t_{o,c} \log(p_{o,c})$$

7.2 Cross-Entropy and Information Theory

Cross-entropy loss derives from the notion of **entropy** $H(X)$, which measures the uncertainty of a probability distribution:

$$H(X) = \begin{cases} - \int_x p(x) \log p(x) dx & \text{if } X \text{ is continuous} \\ - \sum_x p(x) \log p(x) & \text{if } X \text{ is discrete} \end{cases}$$

(Logarithm in base 2.)

- Higher $H(X) \Rightarrow$ greater uncertainty in the distribution.

- A **uniform** $p(x) \Rightarrow$ maximum entropy (maximum uncertainty).
- A **Dirac delta** $p(x) \Rightarrow$ minimum entropy (zero uncertainty).

Minimizing cross-entropy is equivalent to concentrating the predicted distribution on the correct class. Hence: **the lower the loss, the better the model.**

7.3 Loss as an Optimization Objective

Because loss functions are **differentiable**, the learning problem can be cast as an **optimization problem**.

Key Insight

By training the network, we want to find the weights that achieve **minimal loss**. The weight update algorithm is called **Gradient Descent**, implemented in practice via the **backpropagation procedure**.

8 Backpropagation and Gradient Descent

8.1 General Idea

Backpropagation iteratively updates **weights** w and **biases** b to decrease the loss $J(w)$. The procedure decomposes into three sub-tasks:

1. **Forward pass**: compute the network output and evaluate the loss.
2. **Backward pass**: compute gradients $\partial J / \partial w$ by applying the chain rule layer by layer (from output to input).
3. **Weight update**: subtract a fraction of the gradient from each weight. The fraction is controlled by the **learning rate** η — a critical hyperparameter.

The update corresponds geometrically to moving in the direction of **steepest descent** on the loss surface.

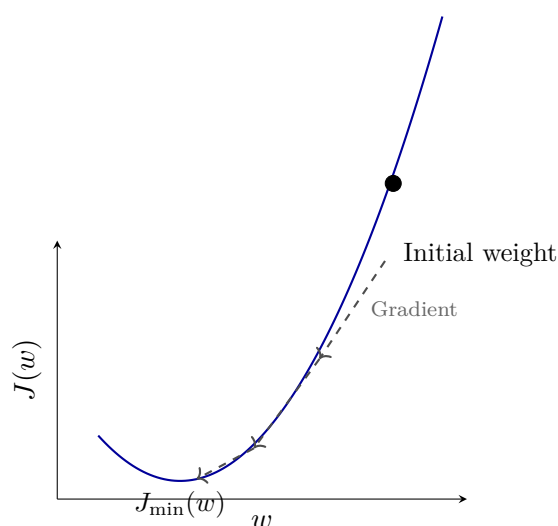


Figure 4: Gradient descent on a 1D loss surface. Each step moves in the direction of the negative gradient toward the global minimum.

8.2 Batch Gradient Descent

1. Initialize weights randomly: $w \sim \mathcal{N}(0, \sigma^2)$.
2. Repeat until convergence:
 - (a) Compute the gradient $\frac{\partial J(w)}{\partial w}$ over the *entire* training set.
 - (b) Update: $w \leftarrow w - \eta \frac{\partial J(w)}{\partial w}$.
3. Return w .

Drawback: computing the gradient over all training samples is very expensive when the dataset contains millions of examples.

8.3 Stochastic Gradient Descent (SGD)

SGD reduces the cost by estimating the gradient from a *single* training sample i at each step:

1. Initialize weights randomly: $w \sim \mathcal{N}(0, \sigma^2)$.
2. Repeat until convergence:
 - (a) Pick a single data point i .
 - (b) Compute the gradient $\frac{\partial J_i(w)}{\partial w}$.
 - (c) Update: $w \leftarrow w - \eta \frac{\partial J(w)}{\partial w}$.
3. Return w .

Drawback: the gradient estimate is very noisy since it relies on a single instance.

8.4 Mini-Batch Gradient Descent

Mini-batch gradient descent is the standard approach in practice — the middle ground between batch GD and SGD:

1. Initialize weights randomly: $w \sim \mathcal{N}(0, \sigma^2)$.
2. Repeat until convergence:
 - (a) Pick a batch B of data points.
 - (b) Compute the averaged gradient:

$$\frac{\partial J_B(w)}{\partial w} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(w)}{\partial w}$$

- (c) Update: $w \leftarrow w - \eta \frac{\partial J(w)}{\partial w}$.
3. Return w .

Mini-batch gradient descent yields **smoother convergence**, **more stable gradients**, and allows for **larger learning rates** compared to pure SGD — making it the default optimization strategy in deep learning.

Part II

Backpropagation: Derivation and Training Dynamics

9 Expressive Capacity of Multilayer Networks

9.1 What a Single Perceptron Can Represent

A perceptron with n inputs defines a hyperplane of dimension $n - 1$ that partitions the input space into two half-spaces:

$$\sum_{i=1}^n w_i x_i - t = 0$$

This limits its classification ability to **linearly separable** problems. The canonical failure case is XOR — no values w_0, w_1, w_2 can satisfy the four constraints simultaneously.

9.2 Capacity with Hidden Layers

Adding hidden layers dramatically increases expressive power. In the two-dimensional case:

- Each hidden neuron defines a boundary line (half-plane).
- With h hidden neurons, the output neuron takes their conjunction: three neurons yield a triangle, four a quadrilateral, and so on.

The following theorems quantify this:

Theorem 9.1 (Huang & Lipmann, 1988). *A feed-forward network with a single hidden layer can approximate almost all separation surfaces between two classes. The qualifier “almost all” means that by adding a second output neuron, non-convex and disconnected regions become representable too.*

Theorem 9.2 (Gibson & Lowan, 1990). *There exist regions that a single-hidden-layer network cannot approximate — for example, two triangular regions that touch at exactly one point.*

Theorem 9.3 (Lipmann, 1987). *A feed-forward network with **two** hidden layers can approximate any region in input space. Approximation accuracy is controlled by the number of neurons in the intermediate layers.*

Remark. These results are existence theorems, not constructive recipes. In practice, there are no fixed rules for choosing architecture; empirical heuristics and cross-validation guide the design.

10 Formal Derivation of Backpropagation

10.1 Setup and Notation

Consider a two-layer feed-forward network with n inputs, h hidden neurons, and q output neurons. Let g be a differentiable activation function (e.g. the logistic). Define:

I_i	i -th input
w_{ji}	weight from input i to hidden neuron j
$a_j = \sum_{i=1}^n w_{ji} I_i$	pre-activation of hidden neuron j
$Z_j = g(a_j)$	output of hidden neuron j
W_{kj}	weight from hidden neuron j to output neuron k
$a_k = \sum_{j=1}^h W_{kj} Z_j$	pre-activation of output neuron k
$O_k = g(a_k)$	output of network for class k

The squared error for a single training example with target y_k :

$$E(W) = \frac{1}{2} \sum_{k=1}^q (y_k - O_k)^2$$

10.2 Derivatives for the Output Layer

We want $\partial E / \partial W_{kj}$. By the chain rule:

$$\frac{\partial E}{\partial W_{kj}} = \frac{\partial E}{\partial O_k} \cdot \frac{\partial O_k}{\partial a_k} \cdot \frac{\partial a_k}{\partial W_{kj}}$$

Computing each factor:

$$\frac{\partial E}{\partial O_k} = -(y_k - O_k), \quad \frac{\partial O_k}{\partial a_k} = g'(a_k), \quad \frac{\partial a_k}{\partial W_{kj}} = Z_j$$

Define the **output delta**:

$$\delta_k := (O_k - y_k) g'(a_k)$$

Then:

$$\boxed{\frac{\partial E}{\partial W_{kj}} = \delta_k Z_j}$$

The weight update (gradient descent) for the output layer is:

$$\Delta W_{kj} = -\eta \delta_k Z_j$$

10.3 Derivatives for the Hidden Layer

We want $\partial E / \partial w_{ji}$. By the chain rule:

$$\frac{\partial E}{\partial w_{ji}} = \frac{\partial E}{\partial a_j} \cdot \frac{\partial a_j}{\partial w_{ji}}$$

The second factor is simply $\partial a_j / \partial w_{ji} = I_i$. For the first factor we must account for the fact that a_j feeds all q output neurons through Z_j :

$$\frac{\partial E}{\partial a_j} = \sum_{k=1}^q \frac{\partial E}{\partial a_k} \cdot \frac{\partial a_k}{\partial Z_j} \cdot \frac{\partial Z_j}{\partial a_j} = \sum_{k=1}^q \delta_k W_{kj} \cdot g'(a_j) = g'(a_j) \sum_{k=1}^q \delta_k W_{kj}$$

Define the **hidden delta**:

$$\delta_j := g'(a_j) \sum_{k=1}^q \delta_k W_{kj}$$

Then:

$$\frac{\partial E}{\partial w_{ji}} = \delta_j I_i$$

The weight update for the hidden layer is:

$$\Delta w_{ji} = -\eta \delta_j I_i$$

10.4 General Weight Update Rule

Both updates share the same structure. For any weight connecting neuron q (input side) to neuron p (output side):

$$\Delta W_{pq} = -\eta \delta_p \cdot V_q$$

where V_q is the activation value at the input neuron and δ_p is the error signal at the output neuron, computed by:

- **Output layer:** $\delta_k = (O_k - y_k) g'(a_k)$
- **Hidden layer:** $\delta_j = g'(a_j) \sum_k \delta_k W_{kj}$ (weighted sum of upstream deltas)

This recursive structure is what gives the algorithm its name: error signals δ are *propagated backward*, from output to input, layer by layer.

10.5 Algorithm

Algorithm 1 Backpropagation (online, one training example per step)

- 1: Initialize all weights randomly (small values).
 - 2: **repeat**
 - 3: **Forward pass:** present input $\mathbf{x}$; compute a_j, Z_j, a_k, O_k for all hidden and output neurons; store activations.
 - 4: **Compute output deltas:** $\delta_k \leftarrow (O_k - y_k) g'(a_k)$ for all k .
 - 5: **Backpropagate:** $\delta_j \leftarrow g'(a_j) \sum_k \delta_k W_{kj}$ for all hidden neurons j .
 - 6: **Update output weights:** $W_{kj} \leftarrow W_{kj} - \eta \delta_k Z_j$.
 - 7: **Update hidden weights:** $w_{ji} \leftarrow w_{ji} - \eta \delta_j I_i$.
 - 8: **until** convergence
-

10.6 Complexity

Computing the gradient via finite differences (difference quotient) costs $\mathcal{O}(W^2)$: for each of the W weights, the network must be evaluated anew. Backpropagation reduces this to $\mathcal{O}(W)$ by reusing intermediate activations and computing all partial derivatives in a single backward sweep. This is the key efficiency gain that makes training large networks tractable.

An additional structural benefit is **locality**: the update ΔW_{pq} depends only on the activation at neuron q and the delta at neuron p — exactly the two endpoints of the connection. This locality makes the algorithm trivially parallelizable.

11 Optimization Challenges

11.1 Learning Rate Sensitivity

The learning rate η is a critical hyperparameter. Its effect is qualitatively different in two failure regimes:

- **η too small:** convergence is slow; training may require an impractical number of epochs.
- **η too large:** the update overshoots the minimum, causing the weights to oscillate between two configurations without converging.

11.2 Local Minima

The loss surface of a deep network is non-convex, with many local minima. Gradient descent is only guaranteed to converge to a *local* minimum; the outcome depends on the initialization. If the starting configuration $W^{(0)}$ is far from the global minimum, the algorithm may settle at a suboptimal solution.

Momentum. A practical remedy is to add a **momentum term** that incorporates inertia from the previous step:

$$\Delta W^{(k)} = -\eta \left. \frac{\partial E}{\partial W} \right|_{W^{(k)}} + \alpha (W^{(k)} - W^{(k-1)})$$

where $\alpha \in (0, 1)$ is the momentum parameter (commonly $\alpha = 0.9$). The effect is to dampen oscillations in directions of high curvature and accelerate movement along low-curvature directions. Other global optimization strategies (simulated annealing, genetic algorithms, reactive Tabu search) can also be applied.

11.3 Saturation

When an activation function is in its saturated regime — inputs far from zero for sigmoid or tanh — its derivative g' is nearly zero. Consequently, $\delta \propto g'$ is nearly zero as well, and weight updates become negligibly small. Training effectively stalls.

Mitigation: initialize weights to small random values. Near zero, the activation functions are approximately linear and their derivatives are large, so gradients are well-conditioned at the start of training. As learning proceeds, weights grow only as needed.

12 Generalization

12.1 Definition

A network's **generalization capacity** is its ability to correctly classify inputs *not* seen during training. Minimizing training loss is a means to an end, not the end itself. A model that perfectly memorizes training examples but fails on new ones has not learned anything useful.

12.2 Overfitting

Overfitting (also called overtraining) occurs when the network has been trained for too many epochs: it has memorized the training patterns, including their noise, at the expense of flexibility on unseen inputs. The telltale sign is a divergence between training and test error:

- **Training error:** monotonically decreasing.

- **Test error:** initially decreasing, then increasing after some epoch e_0 .

The optimal stopping point is e_0 , before the test error begins to rise. This technique is known as **early stopping**.

Key Principle

Minimizing training error is **not** equivalent to training a good neural network. The minimum training error does not correspond to the best generalization. Overfitting must always be monitored.

13 Cross-Validation Strategies

In practice, labeled data is limited. The following strategies manage the training-evaluation tradeoff under this constraint.

13.1 Resubstitution

Train and evaluate on the *same* set (all available data). This produces an **optimistic** (lower-bound) error estimate: the model has already seen every example during training, so measured error is artificially low.

13.2 Holdout

Randomly partition the dataset into two disjoint halves: one for training, one for testing. This provides an **upper-bound** error estimate, since the model sees only half the data during training. The result is sensitive to the particular partition chosen.

13.3 Averaged Holdout

Repeat the holdout procedure multiple times with different random partitions and average the resulting errors. This reduces the variance introduced by a single arbitrary split. Partitions can be sampled randomly or enumerated exhaustively.

13.4 Leave-One-Out (LOO)

Given N examples, train on $N - 1$ and test on the single excluded example; repeat for every example and average. This maximally utilizes the available data and provides a near-unbiased error estimate. The computational cost is N training runs, which can be prohibitive for large datasets or slow-converging models.

13.5 k -Fold Cross-Validation

A practical compromise between holdout and LOO. Partition the data into k disjoint segments (folds) of equal size. For each fold:

1. Train on the remaining $k - 1$ folds.
2. Evaluate on the held-out fold.

Average the k test errors. Leave-one-out corresponds to $k = N$. A common choice in practice is $k = 5$ or $k = 10$, balancing computational cost against variance.

Method	Training size	Bias	Variance
Resubstitution	N	High (optimistic)	Low
Holdout	$N/2$	Moderate	High
Averaged Holdout	$N/2$	Moderate	Lower
Leave-One-Out	$N - 1$	Low	High
k -Fold	$(k - 1)N/k$	Low-moderate	Moderate

14 When to Use Neural Networks

Neural networks are an appropriate choice when:

1. **The problem is instance-based:** examples are given as (attributes, class) pairs. Raw inputs should be preprocessed — real-valued features scaled to $[0, 1]$, discrete features one-hot encoded.
2. **The training set is large:** generalization improves monotonically with data; small datasets favor simpler, more data-efficient models.
3. **Long training times are acceptable:** iterative optimization over large parameter spaces is inherently expensive.
4. **Interpretability is not required:** the learned weight matrices are opaque; if human-readable discriminant functions are needed, decision trees or linear models are preferable.

Part III

Convolutional Neural Networks

15 Regularization

Deep networks have high expressive power and will readily memorize training data. **Regularization** constrains the optimization problem to encourage generalization on unseen inputs, rather than overfitting to training noise.

15.1 The Bias-Variance Trade-off

Model quality is governed by two competing sources of error:

- **Bias:** error from overly simplistic assumptions; the model underfits the data.
- **Variance:** error from excessive sensitivity to training data; the model overfits.

As model complexity increases, bias decreases but variance grows. The optimal model lives at the minimum of their sum. Deep networks are high-variance models by default — regularization is what keeps them on the right side of this trade-off.

15.2 Early Stopping

Stop training as soon as the validation loss stops improving. In practice:

- **Training loss:** monotonically decreasing.
- **Validation loss:** decreases early, then rises as overfitting begins.

The checkpoint at the minimum validation loss is retained as the final model. This is the simplest and most widely used regularizer.

15.3 L2 Regularization (Ridge / Weight Decay)

Add an L2 penalty on all weights to the loss:

$$\mathcal{L}' = \mathcal{L} + \frac{\beta}{2} \|W\|_2^2 = \mathcal{L} + \frac{\beta}{2} \sum_i w_i^2$$

where $\beta > 0$ is the penalty factor. The added term penalizes large weights, forcing the network to distribute the learned signal evenly and discouraging any single connection from dominating. In the gradient update this appears as a constant multiplicative decay on weights at each step, hence the name *weight decay*.

15.4 Dropout

Dropout prevents neurons from *co-adapting* — relying on the presence of specific other neurons and thus becoming brittle. During each training step, each neuron is independently retained with probability p (typically $p \geq 0.5$ for later layers; higher for early layers and input) and dropped (set to zero) with probability $1 - p$.

The effect: each training step is equivalent to training a different sub-network. At test time all neurons are active but their outputs are scaled by p to match the expected activation. This ensemble-like behaviour provides strong regularization at negligible cost.

15.5 Batch Normalization

As training progresses, the distribution of inputs to each hidden layer shifts as the parameters of preceding layers change — a phenomenon called **internal covariate shift**. Batch normalization (Ioffe & Szegedy, ICML 2015) mitigates this by normalizing each feature dimension across the current mini-batch:

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

To preserve representational power, two learnable parameters $\gamma^{(k)}$ and $\beta^{(k)}$ are introduced per feature:

$$y^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)}$$

Batch normalization is inserted between the linear transformation and the activation function. It accelerates training, acts as a mild regularizer, and is practically indispensable in very deep architectures.

16 Convolutional Neural Networks

16.1 Motivation: Receptive Fields and Weight Sharing

The mammalian visual cortex is organized so that individual neurons respond only to stimuli within a restricted region of the visual field — their **receptive field**. Nearby neurons have overlapping receptive fields that tile the entire visual scene. **Convolutional Neural Networks** (CNNs) are loosely inspired by this architecture.

The fundamental observation is that the same local pattern detector (e.g., an edge detector) is useful everywhere in an image — there is no reason to learn a separate detector for each spatial location. CNNs exploit this by **sharing weights** across space: a single filter is convolved with the entire input, producing a **feature map** that encodes where the detected pattern appears.

16.2 The Convolution Operation

A filter (kernel) K of size $k \times k$ is slid over the input with a chosen **stride** s . At each position, the element-wise product of the filter and the corresponding input patch is summed to produce a single output value. For a 2D input X and filter K :

$$(X * K)[i, j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X[i \cdot s + m, j \cdot s + n] \cdot K[m, n]$$

Example 16.1. A 5×5 input convolved with a 3×3 filter at stride 1 produces a 3×3 feature map. The filter is positioned at each valid location, element-wise multiplied with the input patch, and summed:

$$\begin{pmatrix} 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{pmatrix} * \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 4 & 3 & 4 \\ 2 & 4 & 3 \\ 2 & 3 & 4 \end{pmatrix}$$

Output spatial size. For an input of height/width H , filter size k , stride s , and zero-padding p :

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1$$

16.3 Feature Maps and Filters

A convolutional layer applies d different filters to the same input, producing d feature maps stacked along the **depth** dimension (not to be confused with the depth of the network). Each filter learns to detect a distinct local pattern.

- **Shallow filters** (early layers): detect simple patterns such as oriented edges and color gradients.
- **Deeper filters**: combine the activations of previous layers to detect increasingly abstract patterns — textures, object parts, complete objects.

A layer therefore maps an input volume of shape $H \times W \times d_{\text{in}}$ to an output volume of shape $H' \times W' \times d_{\text{out}}$, where d_{out} equals the number of filters.

16.4 Non-linearity

After each convolution, a non-linear activation is applied element-wise. In modern CNNs this is almost universally **ReLU**:

$$g(z) = \max(0, z)$$

ReLU is applied independently to each element of the feature map, setting all negative values to zero. Without non-linearity, stacked convolutions would collapse to a single linear filter.

16.5 Pooling

Pooling layers reduce spatial dimensions, thereby:

- decreasing the number of parameters and computation,
- introducing limited spatial invariance (small translations of the input produce the same output),

- controlling overfitting.

The most common variant is **max pooling**: a $k \times k$ window slides over the feature map with stride s and outputs the maximum value in each window. A 2×2 max pool with stride 2 is the standard, halving both spatial dimensions.

16.6 Fully Connected Layer and Softmax

After a sequence of convolution and pooling operations, the spatial dimensions are small enough that the feature map is **flattened** into a vector and passed through one or more fully connected layers. The final FC layer has as many neurons as there are classes.

For multiclass classification, the output is passed through **softmax**:

$$P(y = j \mid \mathbf{x}) = \frac{e^{z_j}}{\sum_{k=1}^C e^{z_k}}$$

which converts raw scores $\mathbf{z}$ into a proper probability distribution over C classes. Combined with categorical cross-entropy loss, softmax is the standard output for classification CNNs.

16.7 Standard Architecture Pattern

The canonical CNN architecture stacks convolutional blocks followed by fully connected layers:

$$\text{INPUT} \rightarrow [[\text{CONV} \rightarrow \text{ReLU}]^N \rightarrow \text{POOL}]^M \rightarrow [\text{FC} \rightarrow \text{ReLU}]^K \rightarrow \text{FC (softmax)}$$

Typical choices: $N \leq 3$, $M \geq 1$, $K \leq 2$. The convolutional blocks learn spatial features; the FC layers perform classification.

Summary: CNN Layers

Layer	Operation	Purpose
CONV	Sliding dot product with shared filters	Local feature extraction
ReLU	$\max(0, x)$ elementwise	Non-linearity
POOL	Max/avg over spatial window	Downsampling, invariance
BN	Normalize by batch statistics	Training stability
FC	Matrix multiply + bias	Classification / regression
Softmax	Normalized exponential	Probability over classes

17 Landmark Architectures

The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) catalyzed a decade of architectural innovation. The following architectures represent the major milestones, each introducing a design principle that influenced the field.

17.1 LeNet (1998)

Developed by LeCun et al. for handwritten digit recognition on MNIST (28×28 grayscale, 10 classes, 60k training examples). LeNet established the canonical pattern of interleaved convolution and subsampling:

$$\begin{aligned} \text{INPUT } (32 \times 32) &\rightarrow C_1 (6@28 \times 28) \rightarrow S_2 (6@14 \times 14) \rightarrow C_3 (16@10 \times 10) \\ &\rightarrow S_4 (16@5 \times 5) \rightarrow F_5 (120) \rightarrow F_6 (84) \rightarrow \text{OUTPUT } (10) \end{aligned}$$

Filters were 5×5 at stride 1; subsampling was average pooling 2×2 at stride 2; activation functions were sigmoid/tanh.

17.2 AlexNet (2012)

AlexNet (Krizhevsky et al.) won ILSVRC 2012 with a top-5 error of 15.3%, compared to 26% for the runner-up. It was the first deep CNN applied at scale to ImageNet:

- 5 convolutional layers + 3 fully connected layers; 60 million parameters.
- **First use of ReLU** in a competitive CNN — replacing sigmoid/tanh and substantially accelerating training.
- **Dropout** (rate 0.5) on FC layers to control overfitting.
- Data augmentation (random crops, horizontal flips, color jitter).
- Trained on two GTX 580 GPUs for one week.
- Hyperparameters: SGD with momentum 0.9, learning rate 10^{-2} (manually decayed), L2 weight decay 5×10^{-4} , mini-batch 128.

Architecture (for input $227 \times 227 \times 3$):

1. CONV₁: 96 filters, 11×11 , stride 4 $\rightarrow 55 \times 55 \times 96$; MAX POOL 3×3 s2 $\rightarrow 27 \times 27 \times 96$
2. CONV₂: 256 filters, 5×5 , pad 2 $\rightarrow 27 \times 27 \times 256$; MAX POOL $\rightarrow 13 \times 13 \times 256$
3. CONV₃: 384 filters, 3×3 , pad 1 $\rightarrow 13 \times 13 \times 384$
4. CONV₄: 384 filters, 3×3 , pad 1 $\rightarrow 13 \times 13 \times 384$
5. CONV₅: 256 filters, 3×3 , pad 1 $\rightarrow 13 \times 13 \times 256$; MAX POOL $\rightarrow 6 \times 6 \times 256$
6. FC6: 4096 neurons; FC7: 4096 neurons; FC8: 1000 neurons (class scores)

17.3 ZFNet (2013)

Won ILSVRC 2013. Zeiler & Fergus improved AlexNet by:

- reducing the first convolution from 11×11 stride 4 to 7×7 stride 2 (finer spatial resolution early on),
- expanding the number of feature maps in the middle convolutional layers.

ZFNet also introduced **deconvolution-based visualization** to understand which input patterns activate each filter, providing the first systematic tool for inspecting what CNNs learn.

17.4 VGGNet (2014)

Runner-up at ILSVRC 2014 (Simonyan & Zisserman). VGGNet's central thesis: **depth alone, with small filters, drives performance**. Design rules:

- All convolutions are 3×3 at stride 1, padding 1.
- All pooling is 2×2 max pooling at stride 2.

- 16 weight layers (VGG-16) or 19 (VGG-19); 138 million parameters.

Two stacked 3×3 convolutions have the same receptive field as one 5×5 , but fewer parameters and an extra non-linearity. Three 3×3 convolutions match a 7×7 receptive field with a further reduction in parameters. This insight — *replace large filters with stacks of small ones* — became a standard architectural principle.

Limitation: most memory is consumed by early convolutional layers; most parameters reside in the late FC layers (two FC layers of 4096 units each account for the majority of the 138M parameters).

17.5 GoogLeNet / Inception (2014)

Won ILSVRC 2014 (Szegedy et al., Google). The central innovation is the **Inception module**: rather than choosing a single filter size, apply 1×1 , 3×3 , and 5×5 convolutions and 3×3 max pooling *in parallel* on the same input, then concatenate their outputs along the depth dimension.

- 1×1 **convolutions** (bottleneck projections) reduce channel depth before the expensive 3×3 and 5×5 convolutions, dramatically cutting computation.
- 22 layers total; only 5 million parameters (vs. AlexNet’s 60M) — **12× fewer**.
- No large FC layers; replaced by **global average pooling**, eliminating the parameter bottleneck.
- Two **auxiliary classifiers** attached at intermediate layers combat vanishing gradients during training and provide additional regularization.

Later versions (v2, v3, v4) refined the inception blocks with further factorization (e.g., replacing 5×5 with two 3×3 , or $n \times 1$ followed by $1 \times n$) and incorporated batch normalization.

17.6 ResNet (2015)

Won ILSVRC 2015 with 3.57% top-5 error — surpassing human-level performance (5.1%). Developed by He et al. (Microsoft Research). ResNet’s contribution: **residual (skip) connections** that make very deep networks trainable.

The residual block. Instead of learning the mapping $H(\mathbf{x})$ directly, the block learns the **residual** $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$, and the skip connection adds the input back:

$$H(\mathbf{x}) = F(\mathbf{x}) + \mathbf{x}$$

If the optimal transformation is close to the identity, it is easier for the network to push $F(\mathbf{x}) \rightarrow 0$ than to learn a near-identity from scratch. Skip connections also provide a direct gradient path to early layers, alleviating vanishing gradients.

Bottleneck block. For deeper networks, each residual block uses a 1×1 - 3×3 - 1×1 sequence: the first 1×1 reduces channel depth (e.g. $256 \rightarrow 64$), the 3×3 performs spatial convolution, and the second 1×1 restores depth ($64 \rightarrow 256$). This reduces computation by roughly $10\times$ compared to two 3×3 convolutions with the same channel count.

Training details. Batch normalization after every CONV; He (Xavier/2) initialization; SGD with momentum 0.9; initial LR 0.1, divided by 10 at plateaus; weight decay 10^{-5} ; no dropout.

Scale. The ILSVRC-winning model used 152 layers yet trained faster than VGGNet-16 due to efficient bottleneck blocks and the absence of large FC layers.

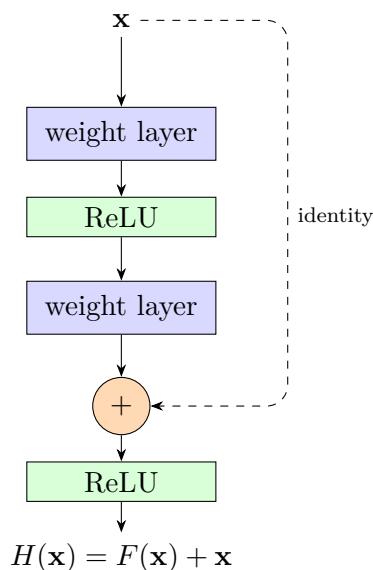


Figure 5: Residual block. The skip connection adds the block’s input directly to its output, making it easy for the block to represent the identity if that is the optimal mapping.

17.7 Performance Summary

Architecture	Year	Layers	Parameters	Top-5 error
AlexNet	2012	8	61M	15.3%
ZFNet	2013	8	62M	11.2%
VGGNet-16	2014	16	138M	7.32%
GoogLeNet	2014	22	5M	6.67%
ResNet-152	2015	152	25M	3.57%
Human expert	—	—	—	5.1%

The trend is clear: more layers consistently improve performance, provided the optimization problem is properly controlled (via residual connections, batch normalization, and regularization). GoogLeNet demonstrates that parameter count and accuracy are not coupled — architectural efficiency matters as much as depth.

18 Transfer Learning

Training a deep CNN from scratch on a new task typically requires millions of labeled examples and weeks of GPU time. **Transfer learning** bypasses this by reusing a network pretrained on a large source dataset (typically ImageNet) as a starting point.

18.1 ConvNet as Fixed Feature Extractor

Remove the last fully connected layer (which encodes source-task class scores) and treat the remaining network as a **fixed feature extractor**. For each input in the new dataset, propagate it forward and extract the activation vector from the penultimate layer (e.g. a 4096-dimensional vector from FC7 in AlexNet). Train a lightweight classifier (linear SVM or logistic regression) on these features. No backpropagation into the pretrained weights is performed.

When to use: new dataset is small and similar to the source dataset. Fine-tuning is risky due to overfitting; the high-level features from the pretrained network are already relevant.

18.2 Fine-tuning

Initialize with pretrained weights, then continue training by backpropagating on the new labeled data. Three common variants:

1. **Fine-tune only the classifier** (last FC layer): equivalent to feature extractor above but trained end-to-end with a very small learning rate. Used when the new dataset is small and similar to the source.
2. **Fine-tune the top few layers**: unfreeze the last 1–3 convolutional blocks in addition to the FC layers. Used when the new dataset is moderately large or somewhat different from the source.
3. **Fine-tune the entire network**: unfreeze all layers and train with a small learning rate. Used when the new dataset is large (low overfitting risk) and substantially different from the source, so source features are not well aligned.

Practical Rule of Thumb

Dataset size	Similarity to source	Strategy
Small	Similar	Linear classifier on CNN codes
Large	Similar	Fine-tune full network
Small	Different	Linear classifier on intermediate features
Large	Different	Fine-tune full network (or train from scratch)

Early layers learn generic features (edges, textures) that transfer broadly; later layers encode increasingly task-specific representations. This is why freezing early layers is almost always safe, regardless of the transfer scenario.

19 Beyond Supervised Classification

19.1 Autoencoders

An **autoencoder** is trained to reconstruct its input:

$$\mathbf{x} \xrightarrow{\text{encoder}} \mathbf{h} \xrightarrow{\text{decoder}} \hat{\mathbf{x}} \approx \mathbf{x}$$

The bottleneck representation $\mathbf{h}$ (latent code) is forced to capture the most informative structure in the data. Autoencoders enable unsupervised feature learning, dimensionality reduction, and anomaly detection. In practice, discriminative tasks tend to yield better embeddings than pure reconstruction.

19.2 Generative Adversarial Networks

A **GAN** (Goodfellow et al., NeurIPS 2014) trains two networks in adversarial competition:

- **Generator** G : maps random noise $\mathbf{z} \sim p(\mathbf{z})$ to synthetic samples $G(\mathbf{z})$.
- **Discriminator** D : classifies inputs as real (from data) or fake (from G).

G tries to fool D ; D tries to distinguish real from generated samples. At equilibrium, $G(\mathbf{z})$ is indistinguishable from real data. The minimax objective is:

$$\min_G \max_D \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

GANs have been applied to image synthesis, style transfer, image-to-image translation (pix2pix, CycleGAN), and data augmentation.

19.3 Recurrent Networks and LSTMs

Recurrent Neural Networks (RNNs) process sequential data by maintaining a hidden state $\mathbf{h}_t$ that is updated at each time step:

$$\mathbf{h}_t = f(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t + \mathbf{b})$$

Vanilla RNNs struggle with long-range dependencies due to vanishing gradients over many time steps.

Long Short-Term Memory (LSTM) networks address this with a gated cell state $\mathbf{c}_t$ that flows through the sequence with only additive interactions — a *conveyor belt* for information. Three learned gates control the information flow:

- **Forget gate:** decides which portions of $\mathbf{c}_{t-1}$ to discard.
- **Input gate:** decides what new information to add to $\mathbf{c}_t$.
- **Output gate:** decides what to expose as the hidden state $\mathbf{h}_t$.

The gating mechanism allows LSTMs to maintain relevant information over arbitrarily long sequences, making them the standard for text, speech, and time-series tasks before the advent of transformers.

19.4 Transformers

Transformers (Vaswani et al., 2017) replace recurrence with **self-attention**, allowing every position in a sequence to attend directly to every other position:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

where Q (queries), K (keys), and V (values) are linear projections of the input. The scaling factor $\sqrt{d_k}$ prevents the dot products from growing large and saturating the softmax.

Multi-head attention runs h attention mechanisms in parallel and concatenates their outputs, enabling the model to attend to different aspects of the input simultaneously.

Key architectural components:

- **Positional encoding:** injects position information (since self-attention is permutation-invariant).
- **Feed-forward sublayer:** applied position-wise after each attention block.
- **Add & Norm:** residual connections and layer normalization after each sublayer.

Transformers are now the dominant architecture for NLP (BERT, GPT) and are increasingly applied to computer vision (Vision Transformer, ViT).

Part IV

Image Classification

20 The Image Classification Problem

20.1 Task Definition and Applications

Image classification is the task of assigning a discrete semantic label $y \in \{1, \dots, C\}$ to an input image $\mathbf{x}$. The label is drawn from a fixed, finite vocabulary of C categories. Applications span a broad range:

- **Medical imaging:** distinguishing benign from malignant tissue (Levy et al., 2016), cell type identification, pathology grading.
- **Scientific discovery:** galaxy morphology classification (Dieleman et al., 2014), satellite imagery analysis.
- **Wildlife monitoring:** individual animal recognition (e.g., Kaggle whale recognition challenges).

Classification also serves as the **canonical building block** for more complex visual tasks:

- **Object detection:** apply a classifier at every candidate bounding box.
- **Image segmentation:** assign a class to every pixel.
- **Image captioning and visual question answering:** build on image-level representations learned for classification.
- **Video understanding:** classify per frame or temporal clip.

20.2 Statistical Learning Framework

The standard framework formalizes image classification as supervised learning:

$$y = f(\mathbf{x})$$

where $\mathbf{x}$ is a feature representation of the image, y is the output label, and f is the **prediction function** to be learned.

Training phase. Given a labeled training set $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$, learn f by minimizing prediction error on the training set.

Testing phase. Given an unlabeled test instance $\mathbf{x}$, predict $y = f(\mathbf{x})$. The learned function must generalize to inputs not seen during training.

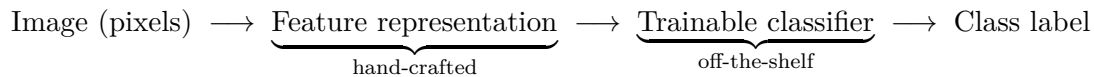
Pipeline. The standard supervised pipeline is:

$$\text{Training images} \xrightarrow{\text{image features}} \text{Training} \rightarrow \text{Learned model}$$

At test time: test image $\xrightarrow{\text{image features}}$ Prediction (using learned model) $\rightarrow$ label.

21 Classic Recognition Pipeline

Before deep learning, recognition systems decoupled feature extraction from classification into two independent stages:



Hand-crafted features had to encode the visual properties that discriminate classes while being robust to nuisance variation (illumination, viewpoint, scale, occlusion). Representations were organized in a hierarchy:

- **Low level:** SIFT (scale-invariant feature transforms), HoG (histogram of oriented gradients), GIST, edge maps.
- **Mid level:** Bag of words (Fisher vectors, VLAD), sliding window detectors, deformable part models.
- **High level:** Contextual dependence, scene-level priors.

The trainable classifier (SVM, logistic regression, random forest) operated on these fixed features. The critical insight from Torralba & Efros is that **three factors** govern recognition performance, roughly in decreasing order of impact:

1. **Data:** more labeled data is almost always better (quality matters more than quantity in the low-data regime). Annotation is the bottleneck.
2. **Representation:** a richer, more invariant feature hierarchy matters more than the choice of classifier.
3. **Learning technique:** the choice of classifier or inference method is the least decisive factor once data and features are fixed.

Remark. Deep learning merges stages 1 and 2: the network learns both the feature hierarchy and the classifier end-to-end from data, replacing hand-crafted features with *deep learned features*.

22 Classification Fundamentals

22.1 Binary and Multiclass Classification

A classifier assigns an input $\mathbf{x} \in \mathbb{R}^d$ to one of C classes. A **decision rule** partitions the input space into **decision regions** — one per class — separated by **decision boundaries**.

- **Binary classification** ($C = 2$): a single decision boundary separates two regions.
- **Multiclass classification** ($C > 2$): the input space is partitioned into C regions. One-vs-one or one-vs-all strategies reduce multiclass to binary; alternatively, a single model can output a C -dimensional score vector.

22.2 Classification Methods

The main families of classification methods, roughly in historical and complexity order:

Method	Characteristic
k -Nearest Neighbours	Non-parametric; decision boundary from local vote
Linear classifiers	Hyperplane boundary; closed-form or convex optimization
Support Vector Machines	Maximum-margin linear classifier; kernel trick for non-linearity
Random Forests	Ensemble of decision trees; handle discrete features naturally
Neural Networks	Learnable non-linear decision boundaries; trained by SGD
Deep Networks	Deep hierarchical feature learning; state of the art on vision

23 Neural Networks for Classification

23.1 Linear Machine

The simplest parameterized classifier maps the input $\mathbf{x} \in \mathbb{R}^d$ to a score vector $\mathbf{s} \in \mathbb{R}^C$ via a linear **score function**:

$$\mathbf{s} = W\mathbf{x}$$

where $W \in \mathbb{R}^{C \times d}$ is the weight matrix. Each row of W is a **class template**: the score for class c is the dot product of $\mathbf{x}$ with the c -th template.

Example 23.1. For a 32×32 RGB image, the input is flattened to $\mathbf{x} \in \mathbb{R}^{3072}$. With $C = 10$ classes, $W \in \mathbb{R}^{10 \times 3072}$ has 30,720 parameters. The output $\mathbf{s} \in \mathbb{R}^{10}$ gives one score per class.

23.2 Two-Layer and Three-Layer Networks

Adding one hidden layer with ReLU non-linearity gives a **2-layer neural network**:

$$\mathbf{s} = W_2 \max(0, W_1 \mathbf{x})$$

where the hidden vector $\mathbf{h} = \max(0, W_1 \mathbf{x})$ has dimension H . The network is also called a *1-hidden-layer network*.

A **3-layer (2-hidden-layer) network** adds a further non-linear stage:

$$\mathbf{s} = W_3 \max(0, W_2 \max(0, W_1 \mathbf{x}))$$

These stacked non-linearities are collectively referred to as **Multi-Layer Perceptrons** (MLPs).

Example 23.2. For CIFAR-10: $d = 3072$, $H = 100$ hidden units, $C = 10$ classes.

$$W_1 \in \mathbb{R}^{100 \times 3072} \quad (100 \times 3072 \text{ parameters})$$

$$W_2 \in \mathbb{R}^{10 \times 100} \quad (10 \times 100 \text{ parameters})$$

Total: $3,072 \times 100 + 100 \times 10 = 308,200$ parameters.

Neural Networks: Pros and Cons

- | | |
|-------------|--|
| Pros | Flexible and general function approximation framework
Scalable depth: adding layers increases model capacity |
| Cons | Hard to analyze theoretically; training prone to local optima
Requires large amounts of training data and compute
Huge hyperparameter space (architecture, learning rate, ...) |

24 Softmax Classifier and Cross-Entropy Loss

24.1 From Scores to Probabilities

The **softmax classifier** (multinomial logistic regression) interprets the raw score vector $\mathbf{s} = f(\mathbf{x}_i; W) \in \mathbb{R}^C$ as **unnormalized log-probabilities** (logits). To obtain a proper probability distribution, apply the **softmax function**:

$$P(Y = k \mid X = \mathbf{x}_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Example 24.1. Given raw scores $\mathbf{s} = [3.2, 5.1, -1.7]$ for classes (cat, car, frog):

class	score (logit)	unnorm. prob. (e^s)	prob.
cat	3.2	24.5	0.13
car	5.1	164.0	0.87
frog	-1.7	0.18	0.00

If the true class is “cat”, the predicted probability is 0.13.

24.2 Cross-Entropy Loss

Training minimizes the **negative log-likelihood** (cross-entropy) of the correct class:

$$L_i = -\log P(Y = y_i \mid X = \mathbf{x}_i) = -\log \frac{e^{s_{y_i}}}{\sum_j e^{s_j}}$$

Continuing the example with true class “cat” ($y_i = \text{cat}$):

$$L_i = -\log(0.13) = 2.04$$

The loss is low when the model assigns high probability to the correct class, and high otherwise. Minimizing L_i is equivalent to **maximum likelihood estimation**: choose W to maximize $\prod_i P(Y = y_i \mid X = \mathbf{x}_i)$.

24.3 Dataset Loss

The full training objective averages per-sample losses over the dataset:

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(\mathbf{x}_i, W), y_i)$$

Training finds $W^* = \arg \min_W L(W)$ by gradient descent:

$$W \leftarrow W - \alpha \frac{\partial L}{\partial W}$$

Remark. Cross-entropy compares the predicted probability vector with the **one-hot** ground-truth vector (1 for the correct class, 0 elsewhere). Perfect predictions yield $L_i = 0$; random predictions over C classes give $L_i = \log C$.

25 CNN Spatial Arithmetic

25.1 Stride and the Integer-Fit Constraint

A filter of size F applied to an input of size N with stride s (no padding) produces an output of size:

$$\left\lfloor \frac{N - F}{s} \right\rfloor + 1 \quad (\text{must be an integer})$$

Example 25.1. $N = 7$, $F = 3$:

$$s = 1 : \frac{7 - 3}{1} + 1 = 5 \quad (\text{valid})$$

$$s = 2 : \frac{7 - 3}{2} + 1 = 3 \quad (\text{valid})$$

$$s = 3 : \frac{7 - 3}{3} + 1 = 2.33 \quad (\text{does not fit — stride 3 cannot tile this input evenly})$$

Not all combinations of N , F , s are valid. In practice, s and F must be chosen so that $(N - F)/s$ is an integer.

25.2 Zero Padding

To prevent the spatial dimensions from shrinking at every convolutional layer, it is common to **zero-pad** the input with p pixels on each border before applying the filter. With padding, the output size generalizes to:

$$\frac{N + 2p - F}{s} + 1$$

Size-preserving padding. To keep the spatial dimensions unchanged at stride $s = 1$, set $p = (F - 1)/2$:

$$\frac{N + 2 \cdot \frac{F-1}{2} - F}{1} + 1 = N$$

This requires F to be odd. Common cases:

F	p (to preserve size at stride 1)
3	1
5	2
7	3

Example 25.2. Input $32 \times 32 \times 3$, 10 filters of size 5×5 , stride 1, padding 2.

$$\text{Output spatial size : } \frac{32 + 2 \cdot 2 - 5}{1} + 1 = 32 \quad \Rightarrow \quad 32 \times 32 \times 10$$

$$\text{Parameters : each filter has } 5 \times 5 \times 3 + 1 = 76 \text{ weights} \Rightarrow 10 \times 76 = 760$$

Remark. Without padding, repeated convolutions shrink the spatial dimensions rapidly. A 32×32 input convolved with 5×5 filters at stride 1 becomes $28 \times 28 \rightarrow 24 \times 24 \rightarrow \dots$. This is too aggressive for deep networks, making zero-padding essential.

25.3 1×1 Convolutions

A 1×1 convolutional layer applies a filter of spatial size $1 \times 1 \times C_{\text{in}}$ at every spatial location. Each filter computes a **C_{in} -dimensional dot product**, mixing channels without touching neighboring spatial positions. This effectively performs a **linear channel projection**: $C_{\text{in}} \rightarrow C_{\text{out}}$ depth-wise at each pixel independently.

Example 25.3. Input $56 \times 56 \times 64$, apply 32 filters of size $1 \times 1 \times 64$:

$$\text{output} = 56 \times 56 \times 32$$

Spatial dimensions are preserved; channel depth is reduced from 64 to 32. Parameters: $32 \times (1 \times 1 \times 64 + 1) = 2,080$.

1×1 convolutions are used as bottleneck projections in GoogLeNet/Inception and ResNet to reduce computation before expensive 3×3 or 5×5 convolutions.

25.4 Translation Equivariance

A convolutional layer is **translation equivariant**: applying a spatial translation to the input and then convolving produces the same result as convolving and then applying the same translation to the output. Formally, if T_δ denotes translation by offset δ :

$$\text{CONV}(T_\delta(\mathbf{x})) \approx T_\delta(\text{CONV}(\mathbf{x}))$$

(exact at interior positions; boundary effects break equivariance at the edges due to padding).

This is a direct consequence of weight sharing: the same filter detects the same pattern regardless of where it appears in the input. Equivariance is distinct from **invariance** (where the output does not change at all with translation, as max-pooling approximately achieves).

25.5 Receptive Field

The **receptive field** of a unit in a feature map is the region of the input image that influences its value. Two factors enlarge the receptive field:

- **Number of layers**: each additional layer increases the effective receptive field.
- **Filter size**: larger filters see more of the input at each layer.

Parameter Efficiency: Stacked Small Filters

Two stacked 3×3 conv layers have the same 5×5 receptive field as a single 5×5 layer, but with fewer parameters and an extra non-linearity:

$$\text{One } 5 \times 5 \text{ layer : } 5 \times 5 + 1 = 26 \text{ parameters per filter}$$

$$\text{Two } 3 \times 3 \text{ layers : } 2 \times (3 \times 3 + 1) = 20 \text{ parameters per filter}$$

This is the design principle behind VGGNet.

Effect of max-pooling on receptive field. Max-pooling with stride > 1 :

- **Increases** the effective receptive field of subsequent layers (each unit now covers a larger input region).
- **Decreases** spatial resolution (fewer feature map positions).
- **Discards** precise spatial information — the exact location of the maximum within a pooling window is lost.

This trade-off is fundamental to CNN design: high-resolution features are needed for localization tasks, while large receptive fields are needed for recognition. Fully convolutional architectures (strided convolutions instead of pooling) can preserve more spatial information.

26 Training Best Practices

26.1 Train / Validation / Test Split

The goal of training is a classifier with good **generalization** to never-before-seen data. The standard protocol enforces three non-overlapping data splits:

1. **Training set**: learn model *parameters* (weights) by minimizing the training loss.
2. **Validation set** (held-out): tune *hyperparameters* (architecture, learning rate, regularization, ...) by evaluating validation performance. Iterate steps 1 and 2.
3. **Test set**: evaluate final model performance *once*. Looking at test performance during development is data snooping and invalidates the evaluation.

Remark. The validation set acts as a proxy for the test set during development. Selecting hyperparameters to maximize validation accuracy *does* use the validation set, so validation accuracy is an optimistic estimate of generalization. The test set, used exactly once, gives an unbiased estimate.

26.2 Training Dynamics

During training, four curves are monitored:

- **Training loss**: decreases monotonically as the optimizer minimizes the objective.
- **Validation loss**: decreases initially, then rises when overfitting begins.
- **Training accuracy**: increases; should reach near 100% for an expressive model on a small dataset.
- **Validation accuracy**: increases initially, then plateaus or decreases.

A growing gap between training and validation loss/accuracy is the diagnostic signature of **overfitting**. The optimal stopping point (early stopping) is the epoch at which validation loss is minimal.

Part V

Recurrent Neural Networks and LSTM

27 Sequential Data and Problem Formulation

27.1 Dealing with Sequential Data

Standard feedforward networks assume i.i.d. inputs with fixed size. Many real-world modalities are instead **sequences**: audio signals, natural language text, time series (IoT sensors, ECG), financial data, video frames. Processing these requires architectures that respect temporal structure.

27.2 Approaches to Sequence Modelling

Two classical statistical approaches exist before neural sequence models:

- **Autoregressive models**: predict X_{t+1} directly from a window of recent observations X_{t-1}, X_t using learned weights w_{t-1}, w_t .

- **Hidden state models:** introduce a latent *hidden* variable that summarises past information; the next observation is generated from the hidden state.

A good sequence model must handle:

- Sequences of **different lengths**.
- Both **short** and **long-term dependencies**.
- **Order** between elements.
- **Parameter sharing** across positions in the sequence.

27.3 Problem Formulations

Taking text as the running example, the same architecture can be instantiated in four regimes:

Regime	Input	Output / Example
One-to-many	single vector	sequence (image captioning)
Many-to-one	sequence	single vector (sentiment analysis)
Direct many-to-many	sequence	aligned sequence (video captioning)
Delayed many-to-many	sequence	offset sequence (machine translation)

27.4 Representing Text: Word Embeddings

ML cannot directly consume raw text; a numerical descriptor is required. The pipeline is:

1. Build a **vocabulary** from the corpus.
2. **Index** each word (e.g. $a \rightarrow 0$, $dog \rightarrow 2$, $walk \rightarrow N$).
3. Represent each word as a vector.

Two representation strategies:

- **One-hot encoding:** a vocabulary-length binary vector with a single 1 at the word's index. Very sparse, high-dimensional, hard-coded, and orthogonal — no semantic similarity between words.
- **Word embeddings:** dense, lower-dimensional vectors *learned from data*. Semantically similar words are close in embedding space (e.g. *happy/sad* cluster together; *dog/cat* cluster together).

27.5 Why Fixed-Window Approaches Fail

Three naive approaches to next-word prediction illustrate the fundamental difficulties:

1. **Small fixed window + one-hot:** concatenate one-hot vectors for the last k words and feed to a classifier. *Problem:* the window may not contain the informative context (long-range dependency).
2. **Bag of all available words:** encode the full preceding context as a single binary vector. *Problem:* word order is discarded (“food was good, not bad” vs. “food was bad, not good” have the same representation).
3. **Large fixed window:** concatenate one-hot vectors for all preceding words. *Problem:* no parameter sharing — knowledge about a word at position i does not transfer to the same word appearing at position j .

The fundamental requirements — variable-length input, order preservation, parameter sharing, long-range dependencies — motivate the RNN.

28 Recurrent Neural Networks

28.1 The Recurrence Relation

An RNN applies the **same function and the same weights** at every time step, maintaining a **hidden state** h_t that summarises all past information:

Definition 28.1 (RNN recurrence).

$$h_t = f_W(h_{t-1}, x_t)$$

where W are shared weight matrices, x_t is the input at step t , and h_{t-1} is the hidden state from the previous step.

The explicit forward-pass equations are:

$$h_t = \sigma(W_{hh} h_{t-1} + W_{xh} x_t) \quad (1)$$

$$y_t = W_{hy} h_t \quad (2)$$

where W_{hh} weights the recurrent connection, W_{xh} weights the input, and W_{hy} maps the hidden state to the output.

28.2 Key Properties

- **Memory:** recurrence adds memory — the decision at step $t - 1$ influences the decision at step t .
- **Causal modelling:** the network models causal relationships between observations.
- **Two sources of input:** the present (x_t) and the recent past (h_{t-1}).
- **Weight sharing:** W_{hh} , W_{xh} , and W_{hy} are shared across all time steps. They act as *filters* that determine how much importance to give to the present input and the past hidden state.

28.3 Unrolling Over Time

An RNN with a self-loop can equivalently be viewed as a sequence of multiple communicating copies of the same network, one per time step. This is called **unrolling** (or unfolding) and makes the computational graph acyclic, enabling standard backpropagation.

28.4 Loss and Backpropagation Through Time (BPTT)

In the many-to-many setting, each step t produces a local loss L_t . The total loss is:

$$L = \sum_{t=1}^T L_t$$

The gradient of L_t with respect to the shared weight W_{hh} requires differentiating through all earlier hidden states (chain rule over time):

$$\frac{\partial L_t}{\partial W_{hh}} = \frac{\partial L_t}{\partial y_t} \frac{\partial y_t}{\partial h_t} \left(\sum_{k=1}^t \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W_{hh}} \right), \quad \frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}}$$

The term $\partial h_t / \partial h_k$ is computed as a **product of consecutive Jacobians** along the time axis, one per step from k to t .

28.5 Gradient Pathologies

This product of Jacobians causes two numerical instabilities:

- **Vanishing gradients:** if many factors are < 1 , the product collapses to zero. Gradient signals from distant time steps vanish, so model weights are updated only with respect to recent effects. Long-term dependencies cannot be learned.
- **Exploding gradients:** if many factors are > 1 , the product diverges. SGD steps become arbitrarily large, destabilising training. A practical fix is **gradient clipping**: if the gradient norm exceeds a threshold τ , rescale it to τ before the update:

$$\Theta_{\text{new}} = \Theta_{\text{old}} - \alpha \nabla_{\Theta} L(\Theta), \quad \text{with } \nabla_{\Theta} \leftarrow \frac{\tau}{\|\nabla_{\Theta}\|} \nabla_{\Theta} \text{ if } \|\nabla_{\Theta}\| > \tau$$

The solution to vanishing gradients requires a fundamentally different cell architecture: **gated cells**.

28.6 RNN Applications

RNNs have been successfully applied to: NLP, language modelling, text generation ($x_{t+1} = o_t$), machine translation, image caption generation, speech recognition, and action recognition in video.

29 Long Short-Term Memory (LSTM)

29.1 Motivation

Proposed by Hochreiter and Schmidhuber (1997), the LSTM addresses the vanishing gradient problem by introducing:

- **Connection weights that may change at each time step** — unlike vanilla RNNs with fixed W_{hh} .
- A **cell state** c_t that accumulates information over long durations, separate from the hidden state h_t .
- **Gated units** that learn *when* to forget old state and when to store new information.

29.2 Gated Cells

A **gate** is a sigmoid layer followed by a pointwise multiplication. The sigmoid outputs values in $[0, 1]$: a value of 0 means “let nothing through”; a value of 1 means “let everything through”. Gates are differentiable, so their parameters are learned end-to-end.

29.3 LSTM Cell State

The LSTM cell maintains two vectors of length n :

- **Cell state** c_t : the long-term memory. LSTM can erase, write, and read from c_t via three corresponding gates.
- **Hidden state** h_t : the short-term memory / output, analogous to the vanilla RNN hidden state.

Gate values are dynamic: they are computed from the current context (h_{t-1}, x_t) at each step, not fixed.

29.4 The Four Steps of an LSTM Cell

Step 1 — Forget. The **forget gate** f_t decides which information to discard from the previous cell state:

$$f_t = \sigma(W_f h_{t-1} + U_f x_t)$$

$f_t \in (0, 1)^n$; element-wise multiplication $c_{t-1} \cdot f_t$ selectively erases components of the cell state.

Step 2 — Store input. The **input gate** i_t decides what new information to write into the cell state, and $\tilde{c}_t$ is the candidate cell content:

$$i_t = \sigma(W_i h_{t-1} + U_i x_t) \quad (3)$$

$$\tilde{c}_t = \tanh(W_c h_{t-1} + U_c x_t) \quad (4)$$

i_t answers “is the new data worth remembering?”; $\tilde{c}_t$ determines how much each component of the cell state should be updated.

Step 3 — Update cell state. The new cell state combines the selectively forgotten old state with the selectively written new content:

$$c_t = c_{t-1} \cdot f_t + i_t \cdot \tilde{c}_t$$

The element-wise multiplication by f_t erases; the addition of $i_t \cdot \tilde{c}_t$ writes.

Step 4 — Output hidden state. The **output gate** o_t filters which parts of the (now updated) cell state are exposed as the hidden state:

$$o_t = \sigma(W_o h_{t-1} + U_o x_t) \quad (5)$$

$$h_t = \tanh(c_t) \cdot o_t \quad (6)$$

$\tanh(c_t)$ squashes the cell state to $[-1, 1]$; o_t gates which components are passed on. h_t is then used to produce the output y_t .

LSTM full equations summary

$f_t = \sigma(W_f h_{t-1} + U_f x_t)$	(forget gate)
$i_t = \sigma(W_i h_{t-1} + U_i x_t)$	(input gate)
$\tilde{c}_t = \tanh(W_c h_{t-1} + U_c x_t)$	(candidate cell)
$c_t = f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t$	(cell state update)
$o_t = \sigma(W_o h_{t-1} + U_o x_t)$	(output gate)
$h_t = \tanh(c_t) \cdot o_t$	(hidden state)

Remark. The cell state c_t flows through the network with only element-wise multiplications and additions — no matrix multiplications along the temporal axis. This additive structure is why gradients do not vanish as severely as in vanilla RNNs.

30 Variants

30.1 Gated Recurrent Unit (GRU)

Introduced by Cho et al. (2014), the **Gated Recurrent Unit (GRU)** simplifies the LSTM by:

- Merging the forget and input gates into a single **update gate** z_t .
- Adding a **reset gate** r_t that controls how much past information to forget.
- Merging the cell state and hidden state into a single hidden state h_t .

The GRU equations are:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \quad (7)$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \quad (8)$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t]) \quad (9)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t \quad (10)$$

The resulting model has fewer parameters than a standard LSTM, is computationally cheaper, and has become increasingly popular in practice.

Reference

K. Cho, B. van Merriënboer et al. (2014). “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation”. *EMNLP 2014*.

30.2 Transformers

The **Transformer** (Vaswani et al., 2017) replaces recurrence entirely with a self-attention mechanism. At each step, the model attends to *all* other positions in the sequence simultaneously, deciding which parts are most relevant. This eliminates the sequential bottleneck of RNNs, enables massively parallel training, and handles long-range dependencies more effectively.

Reference

A. Vaswani et al. (2017). “Attention Is All You Need”. *NeurIPS 2017*. arXiv:1706.03762.

Part VI

Attention Mechanism & Transformers

31 From RNNs to Transformers

31.1 The Historical Context

Before Transformers, NLP tasks — especially **sequence-to-sequence** (seq2seq) tasks such as machine translation — relied on recurrent architectures: LSTMs and GRUs. In the visual domain, convolutions dominated for decades. The **attention mechanism** itself was introduced earlier, through “fast weight controllers” (Schmidhuber, 1992–1997), but it only became central to NLP architectures later.

The Transformer architecture, introduced in 2017 in “Attention Is All You Need” (Vaswani et al.), stands on the shoulders of the encoder-decoder paradigm but departs from it in a fundamental way: **recurrence is eliminated entirely**.

31.2 Problems with RNNs/LSTMs

- **Sequential processing:** inputs are processed one after the other, making it impossible to fully exploit modern GPUs designed for parallel computation.
- **Long-range dependencies:** information degrades as sequence length increases; the hidden state must compress an entire context into a fixed-size vector.
- **Vanishing gradients:** gradients shrink over many time steps, making distant context hard to learn.

31.3 Transformers: Key Advantages

- **No recurrence:** the entire sequence is processed in parallel.
- **Self-attention:** any two tokens can interact directly, regardless of their distance in the sequence.
- **Efficiency:** simple matrix operations that are fully parallelizable on modern hardware.
- **Performance:** state-of-the-art on virtually all NLP benchmarks, and increasingly in vision.

Core Idea

Transformers replace recurrence with **self-attention mechanisms**, which analyze the relationships between *all* elements in a sequence simultaneously. Self-attention assigns different importance (weights) to every token, capturing long-range dependencies efficiently and in parallel.

32 Transformer Architecture

The Transformer is an **encoder-decoder** architecture. Its high-level pipeline for a translation task proceeds as follows:

1. **Pre-processing:** tokenization, embedding, positional encoding.
2. **Encoder:** encodes relations among all input elements, producing contextualized representations.
3. **Auto-regressive decoding:** generated outputs are fed back as additional decoder inputs.
4. **Output generation:** the decoder produces a probability vector over the vocabulary for the next token.
5. **De-tokenization:** convert the predicted logits back into words.

32.1 Encoder–Decoder Structure

Definition 32.1 (Encoder). The encoder receives the entire input sequence and outputs an encoded sequence of the *same length*. Its role is to build rich, contextualized representations of the input.

Definition 32.2 (Decoder). The decoder predicts one token at a time, conditioned on (a) the encoder output and (b) the previously predicted tokens. This generation is **auto-regressive** — output elements are produced sequentially, so there is no parallelization at inference time.

Both the encoder and the decoder are **stacks of N identical layers** (the original paper uses $N = 6$). All encoder layers share the same structure; all decoder layers share the same structure. Each decoder layer receives input from both the immediately preceding decoder layer *and* the top encoder layer.

Example 32.1 (Translation). Input: “How are you?” $\rightarrow$ Encoder produces a matrix representation $\rightarrow$ Decoder iteratively generates: “¿Cómo estás?”

33 Encoder in Detail

33.1 Components

An encoder transforms raw input tokens into **contextualized representations**. Each encoder layer consists of:

1. **Input Embeddings** (bottom encoder only)
2. **Positional Encoding**
3. **Multi-Head Self-Attention** sub-layer
4. **Add & Norm** (residual connection + layer normalization)
5. **Feed-Forward Neural Network** sub-layer
6. **Add & Norm**

33.2 Input Embeddings

Input tokens (e.g., words) are converted into **numerical vectors** using an embedding algorithm. The embedding layer captures **semantic meaning**: semantically similar words map to nearby vectors. Embedding happens only at the bottom-most encoder; all subsequent encoder layers receive the output of the encoder directly below them. Each encoder receives a list of fixed-size vectors (e.g., dimension 512).

33.3 Positional Encoding

The Transformer has no inherent notion of order: without additional information, shuffling the input tokens would produce the same representation. **Positional encoding** injects position/order information by adding a positional vector of the same dimension as the embedding.

The standard formula uses sine and cosine functions of different frequencies:

$$P(k, 2i) = \sin\left(\frac{k}{n^{2i/d}}\right), \quad P(k, 2i + 1) = \cos\left(\frac{k}{n^{2i/d}}\right)$$

where:

- k : position of the token in the input sequence, $0 \leq k < L/2$
- d : dimension of the output embedding space
- n : user-defined scalar, set to 10 000 in the original paper
- i : column index, $0 \leq i < d/2$; each i controls both a sine and a cosine dimension

Each dimension is characterized by a unique frequency and offset. Values range in $[-1, 1]$, effectively representing each position. The output of the positional encoding layer is a matrix in which each row is an embedded word *summed with* its positional information.

Effect of Positional Encoding

After adding positional encodings, two words will be *close* in the embedding space if they are both semantically similar **and** appear at nearby positions. Using combinations of sine and cosine functions allows the encoder to generalize to sequence lengths not seen during training.

33.4 Stack of Encoder Layers

Each encoder layer consists of **two sub-modules**:

1. A **multi-head self-attention mechanism**.
2. A **pointwise feed-forward network**.

Around each sub-module there is a **residual (skip) connection**, followed by **layer normalization**:

$$\text{output} = \text{LayerNorm}(x + \text{SubLayer}(x))$$

Residual connections mitigate the vanishing gradient problem, enabling training of deep models.

34 Self-Attention Mechanism

Definition 34.1 (Self-Attention). Self-attention is a mechanism for creating **arbitrarily long-range weighted dependencies** between tokens in a sequence. For each token being processed, it determines how much focus to place on every other token in the same sequence.

Self-attention relates each word in the input to all other words, allowing the encoder to focus on different parts of the input as it processes each token. For example, when encoding the word “it” in “The animal didn’t cross the street because **it** was too tired”, self-attention can link “it” to “animal” rather than “street”.

34.1 Query, Key, Value

Self-attention operates through three vectors derived from each token’s embedding:

- **Query (Q)**: represents the current token “asking” for relevant context.
- **Key (K)**: represents each token “announcing” what it contains.
- **Value (V)**: the actual content associated with a key; used to construct the output.

When a query and a key are highly aligned (high dot product), the corresponding value is **emphasized** in the output. The query, key, and value vectors are obtained by multiplying the embedding X by three trainable weight matrices:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

34.2 Step-by-Step Computation

1. **Compute Q, K, V** : multiply embeddings by the learned weight matrices W^Q, W^K, W^V .
2. **Compute scores**: take the dot product of the query vector with the key vector of every other token. For token at position 1: $\text{score}_j = q_1 \cdot k_j$.
3. **Scale**: divide scores by $\sqrt{d_k}$ (e.g., $\sqrt{64} = 8$) to obtain more stable gradients.

4. **Softmax**: normalize the scaled scores so they are all positive and sum to 1. The softmax score indicates how much each word will be represented in the output at this position.
5. **Weight values**: multiply each value vector v_j by its softmax score.
6. **Sum**: sum up the weighted value vectors to produce the output z at this position.

34.3 Matrix Form

In practice, all tokens are processed simultaneously using matrix operations. Packing all embeddings as rows of X :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- QK^T is the **score matrix**: entry (i, j) measures how much token i should attend to token j . Geometrically it is an unnormalized cosine similarity.
- Dividing by $\sqrt{d_k}$ prevents the dot products from growing too large in high dimensions, keeping gradients stable.
- Softmax normalizes each row into a valid probability distribution (attention weights).
- Multiplying by V produces the final attended representation.

Key Property

Soft attention is **differentiable**, which makes it directly integrable into end-to-end trained neural networks via backpropagation.

35 Multi-Head Self-Attention

35.1 Motivation

A single attention function might be dominated by the token itself (e.g., z_1 contains mostly information about position 1). **Multi-head attention** extends the model's ability to jointly attend to information from **different representation subspaces** at different positions.

Instead of one attention function, queries, keys, and values are **linearly projected h times** with different learned projections. Each projection defines one **attention head**; all heads operate in parallel, each producing an h -dimensional output.

35.2 Mechanics

With h heads and d_{model} model dimension:

- Each head i has its own weight matrices W_i^Q , W_i^K , W_i^V (randomly initialized, learned during training).
- Each head independently projects the inputs and computes attention, producing Z_i .
- The h output matrices $Z_0, \dots, Z_{h-1}$ are **concatenated** and multiplied by a learned projection matrix W^O :

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \\ \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \end{aligned}$$

The result is a single matrix that captures information from all heads. The original paper uses $h = 8$ heads.

Why Multi-Head?

Multiple heads allow the model to simultaneously focus on different aspects of context: one head might resolve coreference (“it” $\rightarrow$ “animal”), another might capture syntactic relations, and yet another might encode positional proximity.

36 Feed-Forward Network and Normalization

36.1 Pointwise Feed-Forward Network

After the (multi-head) attention sub-layer, the output passes through a **pointwise feed-forward network** — the same network applied independently to each position. In the original paper it consists of:

1. Linear layer
2. ReLU activation
3. Linear layer

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2$$

The output merges with the input via another residual connection, followed by another layer normalization.

36.2 Add & Norm Summary

Each sub-layer (self-attention and feed-forward) in every encoder *and* decoder layer is wrapped with:

$$\text{LayerNorm}(x + \text{SubLayer}(x))$$

This residual + normalization pattern stabilizes training and enables stacking many layers.

37 Decoder in Detail

37.1 Structure

The decoder mirrors the encoder but has **three sub-layers** per layer instead of two:

1. **Masked Multi-Head Self-Attention**
2. **Add & Norm**
3. **Encoder-Decoder (Cross) Attention** — the decoder’s queries attend to the encoder’s keys and values
4. **Add & Norm**
5. **Feed-Forward Network**
6. **Add & Norm**

The decoder operates **auto-regressively**: it begins with a start token and generates tokens one by one, feeding each generated token back as part of its next input, until a special end token is produced.

37.2 Masked Multi-Head Self-Attention

Definition 37.1 (Masking). Masking prevents each decoder position from attending to *future* positions. This is enforced by adding a masking matrix M to the scaled scores before softmax:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V$$

The masking matrix M contains:

- 0 for **allowed positions** (attending to past and current tokens).
- $-\infty$ for **masked positions** (future tokens); after softmax these become 0 attention weight.

Concretely, for a sequence of length 3, the look-ahead mask is:

$$M = \begin{pmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{pmatrix}$$

This ensures that predictions at position t depend only on known outputs at positions $1, \dots, t-1$, making the decoder's generation truly sequential and causal.

37.3 Encoder-Decoder Attention

In this sub-layer, the **queries come from the decoder** (previous decoder layer output), while the **keys and values come from the encoder** (top encoder layer output). This allows each decoder position to attend to all positions in the input sequence, integrating the encoder's rich contextual representations.

37.4 Output Stack

The final decoder layer feeds into:

1. A **linear layer** acting as a classifier projecting to vocabulary size.
2. A **softmax** that converts logits to a probability distribution over the entire vocabulary.

The token with the highest probability is selected (or sampled) as the next output token.

38 Language Pre-Training

Transformers are well-suited to **transfer learning**: pre-train on large unlabeled corpora in an unsupervised manner, then fine-tune on small labeled datasets for specific tasks.

Three broad model families emerge depending on which part of the architecture is retained:

Type	Examples	Use cases
Encoder-only	BERT, RoBERTa, Electra	Text classification, sentiment analysis, sentence similarity, NER
Decoder-only	GPT series, Transformer-XL, DialoGPT	Text generation, machine translation, summarization, chatbots
Encoder-Decoder	T5, BART	Seq2seq tasks: translation, summarization, Q&A

- **Encoder-only models** output hidden states for each token; they do not generate text on their own. They are best for tasks requiring a deep understanding of the full input context.
- **Decoder-only models** generate text sequentially in an auto-regressive fashion. GPT-3 has 175B parameters.
- **Encoder-decoder models** use the full pipeline and excel at transformation tasks where both understanding and generation are required.

Key References

- A. Vaswani et al. (2017). “Attention Is All You Need”. *NeurIPS 2017*. arXiv:1706.03762.
- A. Dosovitskiy et al. (2021). “An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale”. *ICLR 2021*. arXiv:2010.11929.

Part VII

Beyond Supervised Learning: Unsupervised Learning, Clustering & Self-Supervised Methods

39 Types of Learning: A Taxonomy

39.1 Supervision Axes

Learning algorithms can be classified along two independent axes:

- **With teacher vs. without teacher:** does the system receive explicit labels or target signals?
- **Active vs. passive:** does the learner choose which examples to query, or is the data fixed?

This yields four quadrants:

	With Teacher	Without Teacher
Passive	Supervised learning	Unsupervised learning
Active	Active learning	Exploration (RL)

39.2 Motivation for Unsupervised Learning

LeCun’s Cake Analogy

“If intelligence is a cake, the bulk of the cake is **unsupervised learning**, the icing on the cake is **supervised learning**, and the cherry on the cake is **reinforcement learning**.”
— Yann LeCun

The information-theoretic argument: a typical supervised dataset provides only 10–10,000 bits per sample (one label). The raw data itself contains millions of bits. Supervised learning

therefore discards most of the available information. By contrast, unsupervised methods can exploit the *full* data distribution.

Why unsupervised learning is hard: unlike supervised learning, there is no clear objective function derived from labels. The “right” representation is task-dependent and not trivially defined.

Key Goals of Unsupervised Learning

- Learn the underlying structure of the data distribution.
- Build **general-purpose representations** that transfer to downstream tasks.
- Exploit the vastly larger pool of unlabeled data.

40 Self-Supervised Learning

40.1 Core Idea

Self-supervised learning (SSL) is a form of unsupervised learning in which the supervision signal is derived automatically from the data itself — no human annotations are required. The procedure is:

1. Design a **pretext task** whose labels can be generated automatically from the raw data.
2. Train a model to solve the pretext task.
3. Use the learned representation for downstream tasks (transfer learning).

Definition 40.1 (Pretext Task). A **pretext task** is an auxiliary learning objective created from the data structure itself (e.g., predicting the relative position of image patches). Solving the pretext task forces the model to learn semantically useful representations, even though the task itself may not be the end goal.

40.2 Context Prediction (Doersch et al., ICCV 2015)

One of the earliest successful SSL methods for visual representation learning. The key idea: given a central image patch, predict the **relative spatial position** of a surrounding patch (one of 8 possible positions).

1. Sample a random 96×96 patch from an image.
2. Sample one of its 8 neighbours.
3. Train a CNN to classify which of the 8 positions the neighbour occupies.
4. The CNN must understand spatial layout and object semantics to solve the task.

Reference

C. Doersch, A. Gupta, A. Efros. “Unsupervised Visual Representation Learning by Context Prediction”. *ICCV 2015*.

41 Learning by Clustering

41.1 General Framework

A natural approach to unsupervised representation learning: use cluster assignments as pseudo-labels to train a network.

Key Observation

Even a **randomly-initialized CNN** already produces a feature space with non-trivial structure. Clustering in that space yields assignments that, used as pseudo-labels, can bootstrap meaningful representations.

The iterative clustering loop:

1. **Feature extraction**: pass all images through the current CNN; collect feature vectors.
2. **Clustering**: run k -means (or similar) on the feature vectors to obtain k clusters.
3. **Pseudo-label assignment**: assign each image the index of its cluster.
4. **CNN fine-tuning**: train the CNN with cross-entropy loss on the pseudo-labels.
5. Repeat from step 1.

42 Progressive Unsupervised Learning (PUL)

42.1 Problem Setting: Unsupervised Person Re-Identification

Person re-identification (re-ID) is the task of matching images of the same person across cameras with non-overlapping fields of view. It is challenging due to viewpoint, illumination, and appearance changes.

In the **unsupervised** setting: no cross-camera identity labels are available. Only *within-camera* tracklets (sequences of frames) provide weak supervision.

42.2 PUL: Formulation

Progressive Unsupervised Learning (Fan et al., ACM TOMM 2018) addresses this with an iterative self-paced scheme combining k -means clustering with CNN fine-tuning and selective sample inclusion.

Definition 42.1 (PUL Setup). Let $\mathcal{X} = \{x_i\}_{i=1}^N$ be the unlabeled cross-camera dataset, and let $\{\hat{y}_i\}$ be cluster pseudo-labels. The model learns a feature embedding f_θ such that same-identity images cluster together.

The three core optimization objectives:

- (1) **Cluster assignment** — find pseudo-labels $\hat{y}$ by k -means:

$$\hat{y}_i = \arg \min_k \|f_\theta(x_i) - \mu_k\|_2^2 \quad (11)$$

where μ_k is the centroid of cluster k .

- (2) **Sample selection** — include only samples close to their centroid:

$$\mathcal{S} = \{x_i : \|f_\theta(x_i) - \mu_{\hat{y}_i}\|_2^2 \leq \tau\} \quad (12)$$

where τ is a distance threshold. This is the **self-paced** component: start with easy (unambiguous) samples, progressively include harder ones as the model improves.

- (3) **CNN fine-tuning** — minimize cross-entropy on selected samples:

$$\mathcal{L} = -\frac{1}{|\mathcal{S}|} \sum_{x_i \in \mathcal{S}} \log p(\hat{y}_i | x_i; \theta) \quad (13)$$

42.3 Algorithm 1: PUL

Algorithm: Progressive Unsupervised Learning

1. **Initialise** CNN f_θ (pre-trained on a source domain with labels).
2. **Extract features:** $F = \{f_\theta(x_i)\}_{i=1}^N$.
3. **Run k -means** on F ; obtain pseudo-labels $\{\hat{y}_i\}$ and centroids $\{\mu_k\}$.
4. **Self-paced selection:** form $\mathcal{S}$ using the distance threshold τ .
5. **Fine-tune** f_θ on $\mathcal{S}$ with cross-entropy loss.
6. **Increase** τ (include more samples next round) and **decrease** k (merge clusters) progressively.
7. **Repeat** steps 2–6 until convergence.

42.4 Self-Paced Learning Intuition

Definition 42.2 (Self-Paced Learning). **Self-paced learning** is a curriculum strategy where the model is trained first on the easiest (most confident) examples, and progressively trained on harder ones as learning progresses. This prevents early-stage noise from corrupting the learned representation.

In PUL:

- Early iterations: small $\tau \Rightarrow$ only samples very close to centroids are selected $\Rightarrow$ high-confidence pseudo-labels.
- Later iterations: τ is increased $\Rightarrow$ more samples included, including ambiguous ones near cluster boundaries.
- k is decreased progressively: coarse-to-fine clustering, merging related pseudo-identities.

42.5 Semi-Supervised Extension

PUL can be extended to the **semi-supervised** setting when a small number of labeled cross-camera pairs are available. The labeled pairs provide an additional supervised loss term:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{unsup}} + \lambda \mathcal{L}_{\text{sup}}$$

This anchors the feature space, significantly reducing the noise in early cluster assignments.

Reference

H. Fan, L. Zheng, C. Yan, Y. Yang. “Unsupervised Person Re-identification: Clustering and Fine-tuning”. *ACM TOMM* 2018.

43 DeepCluster

43.1 Motivation

DeepCluster (Caron et al., ECCV 2018) scales the clustering-based SSL approach to ImageNet-scale data. The central observation: *clustering and representation learning are mutually reinforcing* — better features yield better clusters, and better pseudo-labels yield better features.

43.2 Pipeline

DeepCluster Pipeline

1. **Feature extraction:** forward-pass the full dataset through the current ConvNet; collect the penultimate-layer (FC2-like) features.
2. **Pre-processing:** apply **PCA** for dimensionality reduction, then **L2 normalisation**.
3. **k -means clustering:** cluster all feature vectors into k clusters (e.g., $k = 10,000$ for ImageNet).
4. **Pseudo-label assignment:** use cluster indices as class labels for the entire dataset.
5. **CNN training:** train the ConvNet with **cross-entropy loss** on the pseudo-labels via standard mini-batch SGD + backpropagation.
6. **Repeat** steps 1–5 for the next epoch.

43.3 Architecture and Implementation Details

- **Backbone:** AlexNet or VGG-16; ImageNet-scale experiments.
- **Number of clusters:** $k = 10,000$ — much larger than the 1,000 ImageNet ground-truth classes; allows finer granularity.
- **Initial label generation:** features from the FC2 layer of the randomly-initialised network are already structurally non-trivial; the first k -means run gives a noisy but non-degenerate starting point.
- **Data augmentation:** random crops and horizontal flips are applied during CNN training (consistent with supervised ImageNet training) to improve generalisation.
- **Trivial solution prevention:** without countermeasures, k -means may collapse features onto a small number of clusters. DeepCluster addresses this by:
 - Re-assigning empty clusters.
 - Uniform sampling within each cluster during mini-batch construction (to counter class-size imbalance).

43.4 Why It Works

Example 43.1 (DeepCluster Feature Quality). After sufficient iterations, the convolutional filters of the DeepCluster-trained AlexNet closely resemble those of a supervised AlexNet — oriented Gabor filters, colour blobs, and higher-level part detectors emerge purely from the clustering signal. The representations transfer competitively to PASCAL VOC classification and detection benchmarks.

- **Iteration 0** (random init): features are unstructured; clusters are noisy.
- **After E epochs:** features encode semantic content; clusters become semantically coherent.
- The self-reinforcing loop drives the network toward semantically meaningful representations without any labels.

Reference

M. Caron, P. Bojanowski, A. Joulin, M. Douze. “Deep Clustering for Unsupervised Learning of Visual Features”. *ECCV 2018*. [arXiv:1807.05520](https://arxiv.org/abs/1807.05520).